Caderno de Exercícios Práticos

Fundamentos de Programação, Algoritmos e Programação Orientada a Objetos

24 EXERCÍCIOS

Instruções do Caderno: Esta lista foi projetada para consolidar os conceitos essenciais de lógica e POO. Resolva cada exercício utilizando a linguagem de sua preferência (ex: JavaScript, TypeScript, Python, Java ou C#).

1 Condicionais (if/else, switch)

Ex 1.1 — Classificação de Triângulos

Crie uma função que receba a medida de três lados de um triângulo e informe se ele é *Equilátero* (3 lados iguais), *Isósceles* (2 lados iguais) ou *Escaleno* (3 lados diferentes). Valide antes se os lados formam um triângulo válido.

Ex 1.2 — Calculadora de Imposto de Renda

Elabore um algoritmo que receba o salário bruto de um funcionário e aplique alíquotas condicionais: até R$ 2.000 (Isento); de R$ 2.001 a R$ 3.500 (10%); acima de R$ 3.500 (20%). Retorne o valor final com desconto.

Ex 1.3 — Menu de Opções com Switch

Construa uma estrutura de menu usando `switch` que receba um código de produto (1 a 4) e imprima a categoria correspondente: 1 - Alimento Não Perecível, 2 - Alimento Perecível, 3 - Vestuário, 4 - Eletrônicos. Retorne "Código Inválido" para outros números.

Ex 1.4 — Sistema de Validação de Acesso

Crie um script de login que verifique o nome de usuário e senha. Se o usuário for `"admin"` e a senha `"1234"`, retorne "Acesso Total". Se o usuário for `"user"` e a senha `"1234"`, retorne "Acesso Limitado". Caso contrário, retorne "Acesso Negado".

2 Laços de Repetição (for, while, do-while)

Ex 2.1 — Gerador de Tabuada Completa

Escreva um programa utilizando a estrutura `for` que receba um número inteiro N e exiba a tabuada desse número de 1 a 10 no formato `N x i = Resultado`.

Ex 2.2 — Sequência de Fibonacci

Utilize um laço `while` para gerar e imprimir os primeiros N termos da sequência de Fibonacci (ex: para N = 7 → 0, 1, 1, 2, 3, 5, 8). O valor de N deve ser fornecido pelo usuário.

Ex 2.3 — Validação de Entrada de Dados

Implemente um laço que solicite repetidamente uma nota ao usuário até que ele digite um valor válido entre 0.0 e 10.0. Caso a nota digitada seja inválida, exiba uma mensagem de erro e repita a leitura.

Ex 2.4 — Filtro e Contagem de Números Primos

Crie um algoritmo com laços aninhados que percorra os números de 1 a 50 e identifique quais deles são números primos, exibindo a contagem total de primos encontrados no intervalo.

3 Funções Simples e Escopo

Ex 3.1 — Conversor de Temperatura

Escreva uma função puramente declarativa `celsiusParaFahrenheit(celsius)` que receba um valor em graus Celsius e retorne o equivalente em Fahrenheit utilizando a fórmula `F = (C * 9/5) + 32`.

Ex 3.2 — Análise de Escopo Global vs. Local

Crie um script contendo uma variável global `taxaServico = 0.10`. Escreva uma função que declare uma variável local com o mesmo nome `taxaServico = 0.15` e calcule o total de uma conta. Imprima os valores dentro e fora da função para comprovar que o escopo local não sobrescreveu o global.

Ex 3.3 — Somatório de Lista com Parâmetro Rest/Variádico

Crie uma função `somarNumeros(...valores)` que seja capaz de receber qualquer quantidade de argumentos numéricos e retorne a soma de todos eles.

Ex 3.4 — Função Utilitária com Shadowing

Demonstre o conceito de *variable shadowing* (sombreamento de variável) criando uma função que utilize o mesmo nome de parâmetro que uma variável externa existente no módulo, explicando através do comportamento da saída qual valor teve prioridade na execução.

4 Tipagem e Inferência de Tipos

Ex 4.1 — Declaração Explicita vs. Inferida

Declare três variáveis usando tipagem explícita (`nome: string`, `idade: number`, `ativo: boolean`). Em seguida, declare outras três variáveis atribuindo valores diretamente sem especificar os tipos e explique como o compilador/interpretador inferiu cada tipo.

Ex 4.2 — Função Estritamente Tipada

Crie uma função `calcularMedia(nota1: number, nota2: number): number` em TypeScript/Java/C#. Tente passar uma string como parâmetro intencionalmente e registre a mensagem de erro que o verificador de tipos emitiu.

Ex 4.3 — Modelagem de Tipos Customizados (Union Types / Enums)

Crie um tipo customizado ou Enum chamado `StatusPedido` com os valores `"Pendente"`, `"Processando"`, `"Enviado"` e `"Entregue"`. Crie uma função que aceite apenas variáveis desse tipo para atualizar o estado de uma compra.

Ex 4.4 — Type Assertions e Conversão Coercitiva

Escreva um código que receba um dado do tipo genérico/desconhecido (`unknown` ou `Object`) contendo um texto numérico (ex: `"250.75"`). Faça o estreitamento de tipo (Type Guard / Coerção) para convertê-lo com segurança para um tipo numérico e aplicar uma operação matemática.

5 Classes e Objetos

Ex 5.1 — Modelagem da Classe Livro

Crie uma classe chamada `Livro` com construtor que receba os parâmetros `titulo`, `autor` e `numeroPaginas`. Instancie pelo menos 2 objetos diferentes dessa classe e exiba suas informações no console.

Ex 5.2 — Classe Conta Bancária Básica

Desenvolva uma classe `ContaBancaria` que receba no construtor o `numeroConta` e o `nomeTitular`, definindo o saldo inicial padrão em 0. Instancie duas contas para titulares distintos.

Ex 5.3 — Vetor/Lista de Objetos

Utilizando a classe `Livro` criada no Ex 5.1, crie um array contendo 4 instâncias de livros. Utilize um laço de repetição para percorrer o array e imprimir o título de todos os livros que possuem mais de 200 páginas.

Ex 5.4 — Comparação de Instâncias e Referências

Crie dois objetos distintos da classe `Livro` contendo exatamente os mesmos dados nos atributos. Faça a comparação de igualdade entre eles (`obj1 == obj2`) e explique por que a avaliação retorna `false` com base no conceito de alocação de memória e referência de objetos.

6 Atributos e Métodos

Ex 6.1 — Métodos de Operação Financeira

Expanda a classe `ContaBancaria` do Ex 5.2. Adicione os métodos `depositar(valor)` e `sacar(valor)`. O método sacar não pode permitir saques maiores que o saldo disponível, exibindo uma mensagem de saldo insuficiente.

Ex 6.2 — Encapsulamento com Modificadores de Acesso / Getters e Setters

Torne o atributo `saldo` da `ContaBancaria` privado. Crie um método getter para consultar o saldo e garanta que ele só possa ser modificado através dos métodos `depositar` e `sacar`.

Ex 6.3 — Atributos e Métodos Estáticos (Static)

Crie uma classe utilitária `CalculadoraGeometrica` com atributos estáticos como `PI = 3.14159` e métodos estáticos como `calcularAreaCirculo(raio)` e `calcularAreaRetangulo(b, h)`, permitindo a chamada direta sem necessidade de instanciar a classe com `new`.

Ex 6.4 — Método Interativo entre Objetos (Transferência)

Adicione um método `transferir(valor, contaDestino)` na classe `ContaBancaria` que receba outra instância de `ContaBancaria` como parâmetro. O método deve debitar o valor da conta de origem e creditar na conta de destino, validando o saldo.

Dica de Estudos: Ao finalizar os exercícios, tente adicionar tratamento de exceções (blocos `try/catch`) para lidar com entradas inválidas ou erros em tempo de execução.